CYBERRESILIENCE CLOUD

Research-informed cybersecurity readiness on an existing cloud-hosted website

MSc Software Engineering · Cloud-Based Web System Design | Taher Amine ELHOUARI · cPanel/PHP deployment edition · 10 October 2026

Evidence status: the cPanel/PHP adaptation passes nine local scoring tests and PHP syntax checks. Public deployment on TaherAmine.org and video capture remain PENDING until the actual upload and verification steps are completed. No public link is asserted without verification.

Executive summary

CyberResilience Cloud is a working, lightweight assessment web application that helps small and medium organizations translate cybersecurity uncertainty into prioritized improvements. Its eighteen-question questionnaire covers the six NIST Cybersecurity Framework (CSF) 2.0 functions: Govern, Identify, Protect, Detect, Respond, and Recover. A server-side PHP 8.1+ scoring API validates the input, calculates six normalized domain scores and an overall readiness indicator, and returns recommendations for the three lowest-scoring domains. The web interface is intentionally limited to HTML, CSS, and JavaScript and is packaged for deployment under the existing TaherAmine.org website on managed Namecheap cPanel hosting. There is no user registration or application-managed assessment history. The delivery package supplies executable code, nine PHP tests, a GitHub Actions workflow, architecture and data-flow diagrams, professional report, slides, and a ten-minute narration script. The implementation is verified locally; the public upload and video recording are described as pending rather than misrepresented as completed.

1. Real-World Problem and Research Context

Cybersecurity guidance is abundant, but the ability to operationalize it is uneven. Small and medium organizations may depend heavily on cloud services, SaaS platforms, outsourced IT, remote work, and digital payment or collaboration systems while lacking a dedicated security architecture, governance team, or formal risk function. This creates a decision problem: leaders need to know which gaps matter most, which actions should be prioritized first, and how to communicate progress without requiring specialist knowledge of hundreds of controls.

NIST's Cybersecurity Framework 2.0 provides a broad outcomes-based structure that is intended to be usable across organization sizes, sectors, and maturity levels. NIST SP 1300 was published specifically as a Small Business Quick-Start Guide for small-to-medium businesses with modest or no cybersecurity plans. That source is directly relevant to this project because it validates the need for a practical entry point rather than a full-scale governance transformation on day one. The six CSF 2.0 functions also offer a useful systems-thinking model: governance shapes direction, identification establishes understanding, protection reduces exposure, detection provides visibility, response limits harm, and recovery restores operations.

CISA's Cross-Sector Cybersecurity Performance Goals reinforce this prioritization approach. CISA describes the CPGs as a baseline set of high-impact cybersecurity practices and explicitly notes their usefulness for small- and medium-sized organizations. The CPGs are also organized around the NIST CSF functions. This supports the project's decision to generate focused improvement actions from the weakest domains instead of merely returning a score. A meaningful assessment should lead to decisions, ownership, and measurable progress.

The project therefore defines the core problem as follows: how can a cloud-based web system convert recognized cybersecurity outcomes into a fast, understandable, privacy-aware self-assessment that helps resource-constrained organizations prioritize improvements? The objective is not to certify compliance or replace a professional security assessment. It is to provide a structured starting point and a repeatable decision-support mechanism.

2. Requirements and Design Goals

The functional requirements were derived from the checkpoint objectives and the research context. The system must present a concise assessment, cover all six CSF 2.0 functions, validate user input, calculate normalized domain and overall scores, categorize maturity, identify weak areas, generate prioritized recommendations, and present results in a form that can be printed or saved as a PDF. The interface must work on desktop and mobile devices and should be understandable to non-specialists.

Non-functional requirements emphasize privacy, understandable scoring, low operational cost, browser compatibility, maintainability, reproducibility and verification. The site must run on existing HTTPS-enabled cPanel hosting without requiring a second SaaS account, cloud access token or database connection. Security requirements include same-origin API requests, no-store scoring responses, strict JSON validation, method restrictions, and defensive browser headers. PHP request execution is stateless; the future database design is documented separately but is not part of the demonstrator. Apache/LiteSpeed serves the static assets, while the hosting PHP runtime executes the scoring service.

A central design goal is explainability. A user should understand why the platform asks the questions, what a score means, and what to do next. The maturity labels - Initial, Developing, Defined, and Adaptive - are deliberately descriptive rather than presented as a formal certification scale. The system makes clear that its role is prioritization and improvement planning.

3. System Architecture

The deployed-site design uses four logical layers. The presentation layer is a responsive HTML/CSS/JavaScript interface, with form controls, progress and results visualization, served beneath /msc/cyberresilience-cloud/ on TaherAmine.org. The application-interaction layer gathers exactly eighteen choices and posts structured JSON to a same-origin API endpoint. The trusted PHP application layer implements /api/score.php and /api/health.php, with reusable scoring logic in /src/scoring.php. Finally, an optional future MySQL relational model documents how explicitly consented organization history could be added; no database is configured in the current version. A scoped .htaccess file adds browser security headers without replacing the existing website configuration.

The demonstrator uses stateless request processing and intentionally avoids storing answers in an application database. This limits exposure of self-reported organizational weaknesses and permits each request to be handled independently. The scoring function does not require file storage, sessions, identity accounts, or cookies. The future schema separates organizations, assessments, and function-level scores and offers indexes for organization/time queries, but is documentation only. Routine hosting access logs may still retain request metadata; avoiding application-managed answer storage is not equivalent to guaranteeing that every network participant has zero metadata.

The delivery workflow distinguishes source control from hosting. The complete application and evidence materials are prepared in a new directory inside the existing MSc GitHub repository, rather than replacing or resetting prior coursework. GitHub Actions can perform PHP syntax checks and the scoring unit tests on new commits. The website deployment is intentionally additive: a release ZIP expands only under public_html/msc/cyberresilience-cloud/, leaving the existing Astro-generated personal site intact. A local smoke test validates behavior before upload; the production acceptance test must be repeated against real public HTTPS URLs after release.

4. Technology Selection and Justification

The frontend uses standards-based HTML, CSS, and JavaScript because the interaction is a concise, accessible questionnaire rather than a large multi-page enterprise application. This avoids unnecessary frontend build dependencies while retaining semantic structure, keyboard focus behavior, responsive layouts and printable results. PHP 8.1+ was selected for the server-side API because it is already supported by the destination Namecheap hosting environment and can serve JSON endpoints without separate Node process management. Pure scoring logic is isolated from the HTTP entry point to keep validation and business logic independently testable.

The chosen hosting path is Namecheap shared cPanel hosting with Apache/LiteSpeed and a PHP runtime under the existing public TaherAmine.org domain. This minimizes project-management overhead and allows the website to use its current SSL, DNS, server runtime and file manager. Deployment is a file-based release under a new route rather than a new domain or cloud subscription. The trade-off is important: conventional shared hosting does not offer the elastic function scaling and managed deployment primitives of dedicated public cloud services such as Vercel, Render, AWS or Azure. It fulfills the public accessibility requirement once live, but the evaluator should confirm that the hosting class is acceptable under the checkpoint’s cloud-platform rubric.

The optional persistence model uses MySQL because it is naturally available through cPanel, not because it is currently required. A future opt-in multi-tenant feature might contain organizations, assessments and scores with foreign keys and time indexes. Implementing such a feature would demand authentication, authorization, tenant isolation, secrets handling, access logging, backups, retention and deletion rules. Excluding the database in the demonstrator is a documented design decision: the app can generate a useful one-time assessment without collecting lasting records of security weaknesses. The SQL schema in the repository is an architecture artifact, not evidence of a deployed data service.

5. Application Implementation

The implemented assessment contains eighteen questions, with three questions mapped to each CSF function. Responses use a five-point scale from 0 (not in place) to 4 (consistent and measured). For each domain, the arithmetic mean of the three responses is normalized to a 0-100 score. The overall score is the arithmetic mean of the six domain scores. This design gives each CSF function equal weight and makes the result easy to explain.

Maturity bands provide an additional interpretation layer. Scores below 40 are categorized as Initial, 40-59 as Developing, 60-79 as Defined, and 80-100 as Adaptive. The three lowest-scoring domains drive the recommendation set. This ensures that the result is not merely descriptive; it points the user toward the areas where improvement would have the greatest effect on balance across the resilience lifecycle.

The interface progresses from an explanation of the problem to a six-section self-assessment and a results panel. It renders an overall score, six graphical bars, explanatory band and three recommended actions. Browser print functionality supports an offline PDF of the result. The client sends the answers to the actual PHP backend; if the API cannot be reached, it shows an explicit error rather than silently calculating in JavaScript. This keeps the reported server-side architecture credible and makes backend deployment failures visible during testing.

6. Security and Privacy Engineering

Security engineering is present at both trust boundaries. The PHP scoring endpoint accepts only POST with application/json, caps request bodies at 8 KB, rejects malformed documents and unknown top-level fields, and validates exactly eighteen unique approved question identifiers with integer values from 0 through 4. It rejects duplicate identifiers, extraneous answer keys and invalid ranges rather than quietly coercing strings to numbers. Responses use no-store headers and are encoded as JSON. The application-level .htaccess adds X-Content-Type-Options, X-Frame-Options, referrer restrictions, camera/microphone/geolocation denials, a content-security policy, and directory-listing protection, subject to the host supporting these Apache modules.

The application follows data minimization: visitors do not create accounts and the scoring module does not store submitted answers to a database, file or session. The API avoids echoing submitted raw answers in error messages, and it treats the self-assessment as transient data. Browser printing is client-initiated; users control whether they save the resulting report locally. No public installation had been verified at the time this document was prepared. Host-level request metadata and operational backups remain under the hosting provider’s separate controls. An optional MySQL schema is present as a future-design illustration only.

OWASP ASVS 5.0 is used as an application-security reference for the development mindset. ASVS provides structured verification requirements spanning architecture, validation and business logic, frontend security, APIs, authentication, authorization, cryptography, secure communication, configuration, data protection, and logging. The current demonstrator intentionally avoids authentication and sensitive persistent data, which reduces attack surface, but a production version with accounts would require systematic verification against relevant ASVS requirements.

7. Hosting Deployment, CI/CD, and Scalability

The release is packaged for the existing TaherAmine.org hosting account. The website archive has public_html/msc/cyberresilience-cloud/ as its root path and does not contain the main site index or root rewrite rules. The webserver serves HTML/CSS/JavaScript from the subdirectory, while PHP processes HTTP requests at score.php and health.php. No external services, persistent database or runtime secrets are necessary. Scalability is bounded by the PHP worker, CPU, memory and request limits of shared hosting; this is appropriate for a short academic demonstration with moderate traffic but must not be described as elastic autoscaling. Moving the stateless API to a supported PaaS is feasible later if required.

The GitHub-ready package contains a scoped Actions workflow at the repository root. On changes within the checkpoint folder, CI runs PHP syntax checks and the pure scoring test suite. The suite exercises nine behavioral categories: minimum and maximum results, intermediate scoring, missing answers, duplicate identifiers, invalid data types and values, unrecognized identifiers or additional fields, weakest-domain ordering, and function/priority counts. Tests run without a database or third-party dependencies. Broader production assurance would add integration-level HTTP assertions, browser automation, accessibility evaluation, load testing, error monitoring and vulnerability scanning. Until the workflow has been pushed and executed, successful CI is not claimed.

A future retained-history design could scale independently by using a managed MySQL database or a managed cloud relational service with connection limits, encrypted backups and retention rules. The sample schema already models organization-to-assessment and assessment-to-score relationships, but it is not connected to the public questionnaire. For the current scope, there is no database query latency, schema migration or credential management on the hot path. Operational continuity for shared hosting would require a provider-aware backup and restore plan, uptime checks and documented recovery procedures; these remain proposed improvements rather than validated attributes of the current release.

8. Evaluation and Results

Local evaluation was performed against functional and security requirements. The delivered code presents eighteen labeled response controls, calculates explainable outcomes using six domains, identifies three low-scoring areas, prints the result and exposes a health endpoint. The PHP scoring module passes nine automated tests, checking boundary scores, a developing score, incomplete or duplicate answers, invalid types, unauthorized identifiers, priority ordering and domain completeness. All PHP source files also pass php -l syntax validation under PHP 8.4. The production-host acceptance test still needs to be run once the cPanel release files have been uploaded; local test success is not evidence of public reachability.

Architecturally, the implementation includes separated UI and PHP business logic, restricted JSON API endpoints, testable stateless scoring, a documented future data model, deployment-scoped security headers, release packaging and path-filtered continuous integration. The results intentionally do not make claims of a finished enterprise product: the hosting server may behave differently from the local PHP environment and service responsiveness depends on the shared provider. Visible API errors prevent a broken deployment from falsely appearing successful by returning unverified client-side computations.

From a UX perspective, the system minimizes terminology overload, keeps the response scale consistent across all questions, groups interpretation around familiar resilience functions, and converts the weakest domains into concrete actions. The responsive layout collapses multi-column sections for smaller screens. The use of text labels in addition to visual bars avoids relying only on color to communicate scores.

9. Limitations and Future Development

The current assessment is deliberately concise. Eighteen questions cannot provide the assurance of a formal audit, penetration test, regulatory review, or detailed control assessment. The equal weighting of the six functions is transparent and easy to understand, but a future version could allow sector-specific weighting, applicability logic, evidence uploads, confidence ratings, and organization profiles. Recommendation logic could also map to more granular CSF outcomes and CISA CPG actions.

A production platform could add authenticated multi-tenant workspaces, historical trend analysis, evidence management, role-based access control, exportable executive reports, and collaboration between business and security owners. PostgreSQL persistence would then be enabled with explicit consent, encryption and retention controls. A further research direction would compare whether organizations improve implementation rates more effectively when recommendations are presented as prioritized business outcomes rather than as control lists.

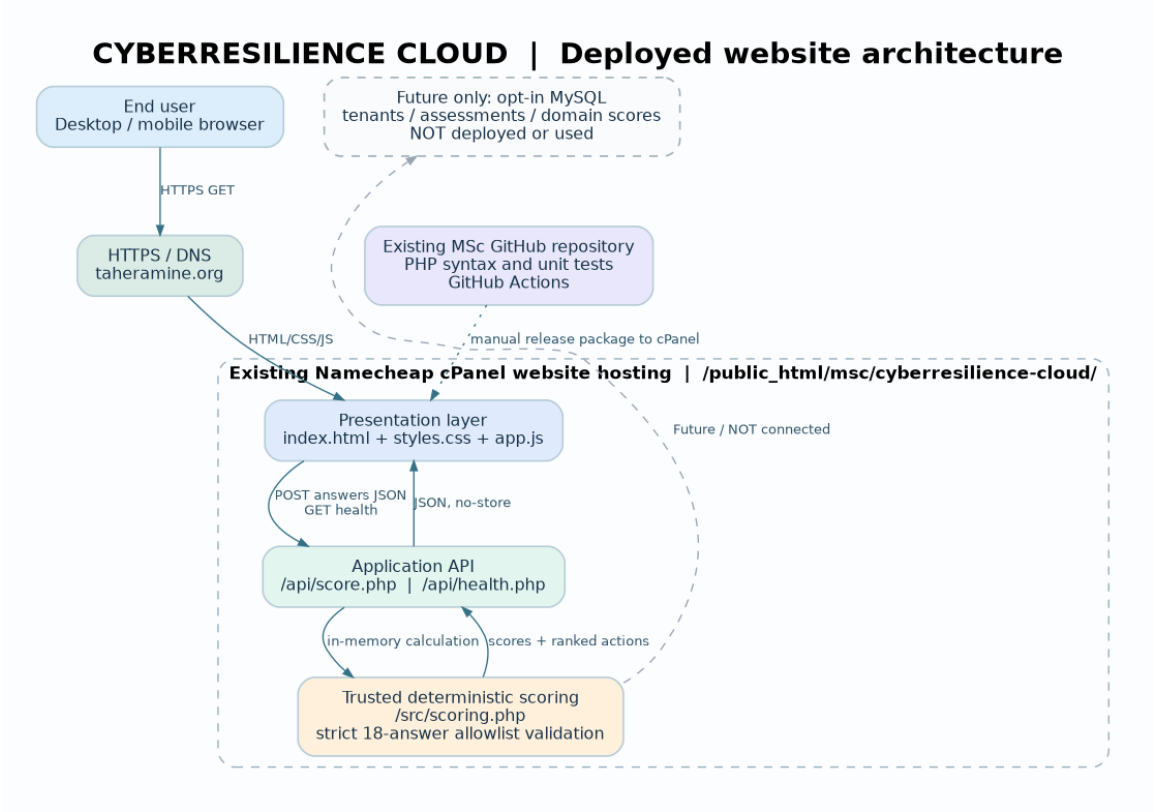
Future resilience enhancements include reliable uptime checks, error dashboards, rate controls at the hosting edge, accessibility audits, additional usability research, typed API contracts, browser-based regression tests and service-availability alerts. Larger organizations or sustained high-volume traffic would justify moving to a platform with managed runtime autoscaling, deployment previews and versioned release rollback. If database history is enabled, additional backup testing, encryption, tenant isolation, lifecycle management and recovery objectives would become mandatory.

10. Conclusion

CyberResilience Cloud combines evidence-led problem definition, software design, risk-aware engineering and a practical web implementation. NIST CSF 2.0 and CISA guidance shape an accessible self-assessment and the recommendation model translates weaknesses into actions. The implemented solution uses a responsive browser interface, stateless PHP backend, explicit request validation and a cPanel-ready release that can be placed beneath the existing website without disrupting it. The project also includes repeatable test cases, source-control packaging, architecture documentation and a future relational data model. Public deployment verification and video recording remain outstanding until independently observed.

The most important design decision is that the application does not treat maturity scoring as the end product. The useful output is the prioritized roadmap that follows. By identifying weak domains and turning them into ownership-oriented actions, the system supports the practical purpose of cybersecurity governance: making better risk decisions and improving resilience over time.

Figure 1 — Executable architecture and trust boundary



The diagram distinguishes implemented presentation and PHP scoring services from the dashed optional MySQL database extension. Transient questionnaire answers cross the HTTPS browser-to-API trust boundary in a JSON POST request; the PHP scoring module validates them, calculates scores in memory, and returns a no-store JSON response without writing an assessment record.

Implementation evidence and verification protocol

The repository package contains a web/ directory with index.html, styles.css, app.js, api/score.php, api/health.php, src/scoring.php, a scoped .htaccess file and no third-party runtime dependencies. The tests/ directory contains the PHP scoring tests. The root GitHub Actions workflow watches changes under Cloud-Based-Web-System-Design/CyberResilience-Cloud and checks syntax and behavior. Local testing requires PHP 8.1 or newer: run php tests/test_scoring.php and php -S 127.0.0.1:8000 -t web, then open the local browser. The API accepts exactly one answers array with eighteen approved records and values limited to integers 0–4; any duplicate, unknown or malformed field is rejected. Function scoring is deterministic and priority tie order is stable.

The local PHP adaptation was checked on 10 October 2026. Nine of nine scoring tests passed and PHP syntax validation succeeded. The acceptance path is then to visit the actual public web URL, open the /api/health.php JSON endpoint, complete the questionnaire or fill the sample, calculate a response with six functions and three priorities, and print to PDF. Screenshots should record the live URL and representative results. Only these public checks can establish deployment evidence; the current local test log cannot establish public accessibility, successful GitHub Actions execution, hosting uptime, performance targets or independent penetration testing.

Delivery blockers and resolution plan

The evaluator's original feedback indicated that only a PDF filename was visible. This is a submission-evidence problem as well as a software-delivery problem. The revised package separates two uploads: a cPanel archive expanding only inside the existing website's msc/cyberresilience-cloud directory, and a new project directory committed into MrTaherAmine/MSc-CS-SWE while preserving all previous coursework. A directly connected deployment attempt to Vercel previously returned 403 Forbidden, so the chosen route avoids that account permission blocker. No cPanel or GitHub publishing permissions are available to this drafting environment. Accordingly the actual site URL, repository commit, CI result, screenshots and demo recording must be verified and added to info.md after the user carries out the uploads. Until then, the live-deployment and recorded-demonstration criteria are incomplete.

Ethical and operational boundaries

Security self-assessments can reveal commercially sensitive control weaknesses. The demonstrator is therefore anonymous and minimizes exposure: answers are submitted only for transient scoring and are not saved by application code. Hosting providers and transport infrastructure may still record request metadata; the visitor's browser may retain local history or printed output. If an authenticated version is later built, it must add a privacy notice, consent where applicable, tenant-safe authorization, key management, encryption, retention policies, data deletion and audit trails. Most importantly, the 0–100 readiness scores are pedagogical indicators, not official NIST maturity ratings, CSF Implementation Tiers or legal-compliance results.

References

NIST. (2024). The NIST Cybersecurity Framework (CSF) 2.0. NIST CSWP 29. https://doi.org/10.6028/NIST.CSWP.29

NIST. (2024). NIST Cybersecurity Framework 2.0: Small Business Quick-Start Guide. NIST SP 1300. https://doi.org/10.6028/NIST.SP.1300

Cybersecurity and Infrastructure Security Agency. Cross-Sector Cybersecurity Performance Goals. https://www.cisa.gov/cybersecurity-performance-goals

OWASP Foundation. Application Security Verification Standard 5.0.0. https://owasp.org/www-project-application-security-verification-standard/